Lab Assignment-4

Implementation and Analysis of Disk Scheduling Algorithms

Course: Fundamentals of Operating System Lab (ENCA252)

1. Objective

To implement and analyze various disk scheduling algorithms — FCFS, SSTF, SCAN, and C-SCAN — using Python on Ubuntu/Linux. The goal is to simulate disk head movements and calculate total seek time for performance comparison.

2. Tools Used

- Operating System: Ubuntu (Linux)
- Language: Python 3.x
- Editor: GNU nano
- Terminal: bash shell

3. Algorithms Implemented

3.1 FCFS (First Come First Serve)

Services disk requests in the order they arrive. Simple but inefficient as head movement is not optimized.

3.2 SSTF (Shortest Seek Time First)

Selects the request nearest to the current head position. Reduces seek time but may cause starvation of far-away requests.

3.3 SCAN (Elevator Algorithm)

Head moves in one direction servicing all requests until reaching the end, then reverses. Balanced performance, avoids starvation.

3.4 C-SCAN (Circular SCAN)

Head moves in one direction only, servicing requests, then jumps back to the beginning without servicing on the return. Provides uniform wait time.

4. Python Source Code (disk_scheduling.py)

```
# OS Lab Assignment-4: Disk Scheduling Algorithms

def fcfs(requests, head):
    seek_time = 0
    sequence = [head]
    for req in requests:
        seek_time += abs(head - req)
        head = req
        sequence.append(req)
    print("FCFS Sequence   :", " -> ".join(map(str, sequence)))
```

```
    print("FCFS Seek Time  :", seek_time)
    return seek_time


def sstf(requests, head):
    seek_time = 0
    sequence = [head]
    reqs = requests.copy()
    while reqs:
        nearest = min(reqs, key=lambda x: abs(x - head))
        seek_time += abs(head - nearest)
        head = nearest
        sequence.append(nearest)
        reqs.remove(nearest)
    print("SSTF Sequence   :", " -> ".join(map(str, sequence)))
    print("SSTF Seek Time  :", seek_time)
    return seek_time


def scan(requests, head, disk_size):
    seek_time = 0
    sequence = [head]
    left = sorted([r for r in requests if r < head], reverse=True)
    right = sorted([r for r in requests if r >= head])
    for r in right:
        seek_time += abs(head - r)
        head = r
        sequence.append(r)
    if left:
        seek_time += abs(head - (disk_size - 1))
        head = disk_size - 1
        sequence.append(head)
        for r in left:
            seek_time += abs(head - r)
            head = r
            sequence.append(r)
    print("SCAN Sequence   :", " -> ".join(map(str, sequence)))
    print("SCAN Seek Time  :", seek_time)
    return seek_time


def cscan(requests, head, disk_size):
    seek_time = 0
    sequence = [head]
    left = sorted([r for r in requests if r < head])
    right = sorted([r for r in requests if r >= head])
    for r in right:
        seek_time += abs(head - r)
        head = r
        sequence.append(r)
    if left:
        seek_time += abs(head - (disk_size - 1))
        sequence.append(disk_size - 1)
        seek_time += (disk_size - 1)
        head = 0
        sequence.append(0)
        for r in left:
            seek_time += abs(head - r)
            head = r
```

```
            sequence.append(r)
    print("C-SCAN Sequence :", " -> ".join(map(str, sequence)))
    print("C-SCAN Seek Time:", seek_time)
    return seek_time


def main():
    print("===== Disk Scheduling Algorithm Simulator =====\n")
    n = int(input("Enter number of disk requests: "))
    print("Enter", n, "disk requests (space-separated): ", end="")
    requests = list(map(int, input().split()))
    head = int(input("Enter initial head position: "))
    disk_size = int(input("Enter disk size (e.g., 200): "))

    print("\n----- Results ----- \n")
    f = fcfs(requests, head); print()
    s = sstf(requests, head); print()
    sc = scan(requests, head, disk_size); print()
    cs = cscan(requests, head, disk_size); print()

    print("===== Performance Comparison =====")
    results = {"FCFS": f, "SSTF": s, "SCAN": sc, "C-SCAN": cs}
    for algo, t in results.items():
        print(algo, ":", t)
    best = min(results, key=results.get)
    print("\nBest Algorithm :", best, "(Lowest Seek Time =", results[best], ")")


if __name__== "__main__":
    main()
```

5. Sample Input & Output

Input Used:

- Number of disk requests: 8
- Request queue: 176 79 34 60 92 11 41 114
- Initial head position: 50
- Disk size: 200

Terminal Output:

```
harshit_18@LAPTOP-P9G31I42:~/disk_scheduling$ python3 disk_scheduling.py
===== Disk Scheduling Algorithm Simulator =====

Enter number of disk requests: 8
Enter 8 disk requests (space-separated): 176 79 34 60 92 11 41 114
Enter initial head position: 50
Enter disk size (e.g., 200): 200

----- Results -----

FCFS Sequence   : 50 -> 176 -> 79 -> 34 -> 60 -> 92 -> 11 -> 41 -> 114
FCFS Seek Time  : 510

SSTF Sequence   : 50 -> 41 -> 34 -> 11 -> 60 -> 79 -> 92 -> 114 -> 176
SSTF Seek Time  : 204
```

```
SCAN Sequence    : 50 -> 60 -> 79 -> 92 -> 114 -> 176 -> 199 -> 41 -> 34 -> 11
SCAN Seek Time   : 337

C-SCAN Sequence : 50 -> 60 -> 79 -> 92 -> 114 -> 176 -> 199 -> 0 -> 11 -> 34 -> 41
C-SCAN Seek Time: 389

===== Performance Comparison =====
FCFS : 510
SSTF : 204
SCAN : 337
C-SCAN : 389

Best Algorithm : SSTF (Lowest Seek Time = 204 )
```

6. Performance Comparison

Algorithm	Total Seek Time	Rank	Remarks
FCFS	510	4	Worst - no optimization
SSTF	204	**1 (Best)**	Lowest seek time
SCAN	337	2	Balanced - elevator
C-SCAN	389	3	Uniform wait time

7. Result Analysis & Conclusion

From the output, SSTF produced the lowest total seek time (204) for the given input, making it the most efficient algorithm in this specific test case. FCFS recorded the highest seek time (510) because it does not optimize head movement. SCAN (337) and C-SCAN (389) performed better than FCFS by sweeping the disk systematically.

Key Observations:

- FCFS is simplest but has the worst performance.
- SSTF offers the best seek time but may cause starvation.
- SCAN balances performance and fairness (elevator-style).
- C-SCAN gives more uniform wait times, good for heavy loads.

Conclusion: No single algorithm is universally best. The choice depends on workload characteristics — SSTF for performance, SCAN/C-SCAN for fairness, and FCFS where simplicity is required.

8. Submission Details

- Name: Sohail khan (2401201080)
- Platform Used: Ubuntu (Linux)
- Language: Python 3
- Date: 2026-04-21